

Predicting Short-Term CPU Thermal Events from System Telemetry

Authors

July 11, 2026

Abstract

We try to predict whether a laptop’s CPU will hit a distress temperature (75 °C) in the next tone minute, using only telemetry the machine already reports: temperatures, power draw, CPU and GPU activity, clock frequencies, memory, and load. Data comes from two Apple Silicon machines (an M1 and an M5), logged every five seconds while a stress script pushed them through repeated heat-and-cool cycles.

The pipeline cleans and imputes the raw logs, builds the label from a forward-looking window, engineers lag and rolling features, and splits the data by session so that no thermal event is seen during both training and testing. We compared logistic regression, random forest, gradient boosting, and a small neural network, with and without corrections for class imbalance.

Gradient boosting won on validation ($F_1 = 0.35$), but its test performance is poor: it catches 5 of the 108 thermal events in the held-out sessions ($F_1 = 0.07$). It does beat a persistence baseline, though only barely, and its ROC-AUC of 0.75 says the features do carry real information. Our reading is that the dataset, not the choice of model, is what is holding us back. Positives are concentrated in a few sessions, and the test sessions are much quieter than the training ones.

1 Problem and Setup

Modern systems throttle silently when they overheat. So, we wanted to see whether a model could see it coming in advance, using nothing but the telemetry the operating system already exposes.

We set this up as binary classification. For every sample, $Y = 1$ if the CPU die temperature reaches 75 °C at some point in the following two minutes, and $Y = 0$ otherwise. Samples near the end of a logging session get dropped, because we cannot see far enough ahead to know whether an event happened, and calling them negatives would have been wrong.

Two things about this problem shaped almost every later decision. The first is that CPU temperature is very autocorrelated. Where the temperature is now tells you a lot about where it will be in a minute, which means a model can look accurate while having learned nothing useful. Because of this we always compare against a persistence baseline, described in Section 6. The second is that thermal events are rare and come in bursts, tied to whatever the machine happened to be doing. If we split the data randomly by row, the same event would end up in both training and test, and our scores would not be valid. Everything is therefore split by session.

2 Data

The machine is logged every five seconds. Each sample records thermal pressure, die temperatures, performance, efficiency-core activity, frequency, CPU and GPU power, memory use, and load averages. Logging runs are grouped into *sessions*, which are just continuous stretches of recording, and sessions are the unit we

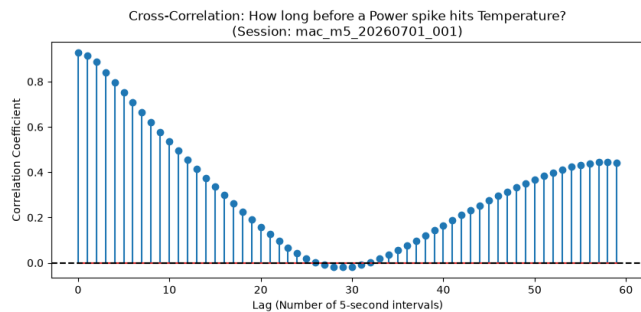


Figure 1: Class balance after labelling: XXX negatives against XXX positives.

use for splitting later. A stress script (optional, but we used it for most sessions) drives bursts of synthetic load with cool-downs in between, so that the machine repeatedly climbs toward the threshold instead of sitting at a constant temperature.

We currently have XXX sessions and XXX raw samples, six from the M1 and seven from the M5. After cleaning and labelling, XXX samples remain, and XXX% of them are positive (Figure 1).

3 Cleaning and Exploration

3.1 Missing data

We trim the first and last few samples of each session, since the logger is still settling down or shutting off, and we forward-fill short gaps. Imputation only ever carries values forward, never backward, so nothing from the future can leak into a past row.

The awkward part is that CPU die temperature,

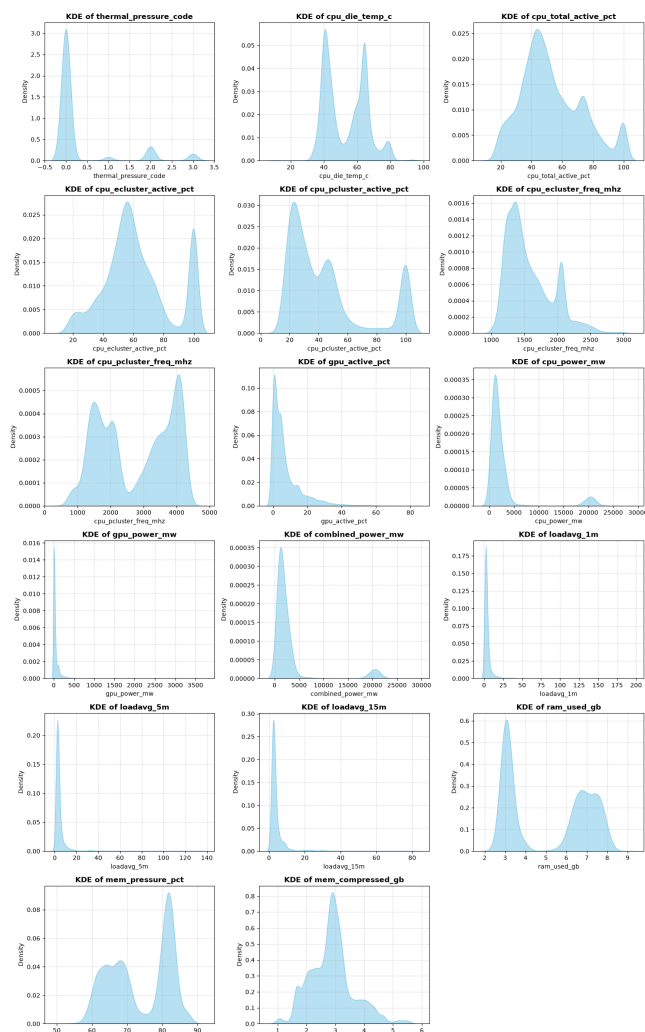


Figure 2: Densities of the main numeric variables.

which is what the label is built from, is missing in 16.8% of rows. Dropping those rows would have cut out large chunks of the stress sessions, which are exactly the parts we care about, so we imputed them instead and kept a flag marking which values were reconstructed. This does mean some of our labels rest on imputed temperatures. This is specifically listed as a limitation in Section 8, but the alternative seemed worse.

3.2 What the variables look like

Figure 2 shows the densities. CPU temperature is clearly multimodal: there is an idle bump around 48 °C, a working bump in the high fifties, and a long tail running past 90 °C. Power draw is heavily skewed to the right, because the machines spend most of their time doing very little. Clock frequencies pile up at the discrete steps the hardware actually uses. RAM usage is bimodal, which is not interesting in itself. It is just that the two machines have different memory footprints and we pooled them.

The practical upshot is that we scale the multimodal columns per mode rather than globally, and power-transform the skewed ones. Standardising everything

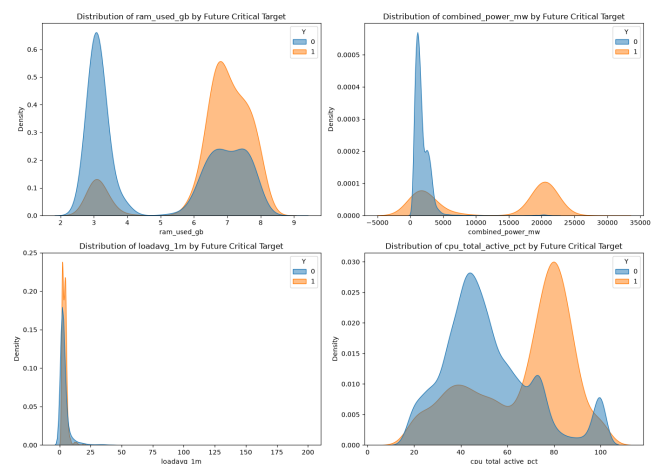


Figure 3: Main variables split by class. Samples before a thermal event sit higher in temperature, power, and P-core activity.

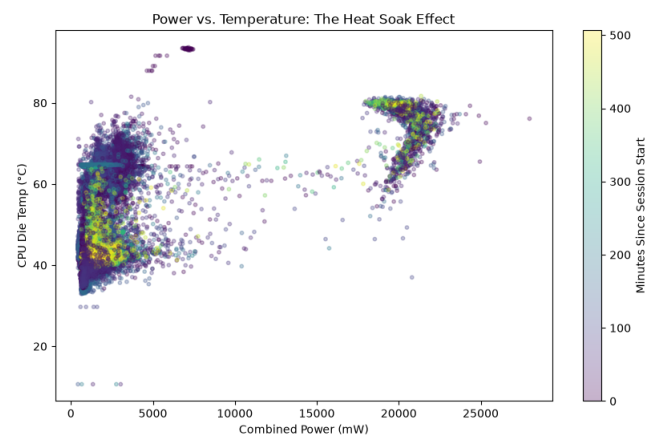


Figure 4: Correlations between the main variables and with Y.

with a single mean and variance would have smeared the idle and stress regimes together, and that is the one distinction the model most needs to see.

3.3 How they relate to the target

Figure 3 splits each variable by class. Samples that precede an event are hotter, draw more power, and show more performance-core activity, which is about what you would expect.

Now, looking at Figure 4, the temperature has the strongest linear relationship with the target, but the power and activity variables are correlated with each other at around $r = 0.9$. That level of collinearity makes logistic regression coefficients unstable and hard to interpret, and it is part of why the tree models later do better.

3.4 Time structure

The ACF in Figure 5 barely decays over the two and a half minutes plotted. This is just thermal inertia, and it is the reason we moved the prediction horizon from one minute out to two. At one minute you could do

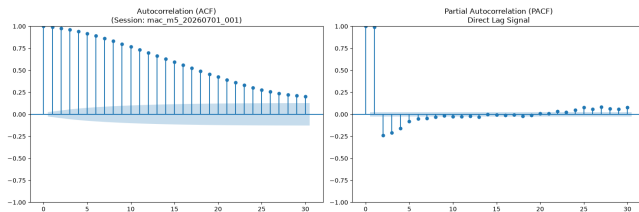


Figure 5: Autocorrelation of CPU temperature out to lag 30 (150s).

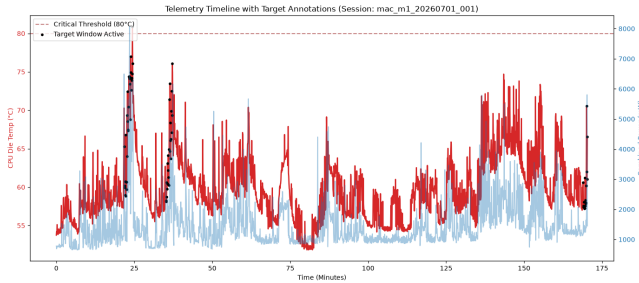


Figure 6: One stress session. The temperature repeatedly climbs across the 75 °C line, and those climbs are what we are trying to anticipate.

well by assuming the temperature simply stays put. At two minutes that trick stops working (see the persistence numbers in Section 6), and the model has to actually anticipate something.

Figure 6 shows a session as the model sees it, with repeated climbs toward the threshold.

4 Features

A single sample tells you where the machine is, not where it is going, so we expand five core signals (CPU temperature, CPU power, combined power, P-core activity, P-core frequency) into features that describe recent history: lagged values, rolling means and standard deviations over windows from seconds to minutes, rates of change, and a couple of hand-made interaction terms. Features are computed within a session, never across the boundary between two.

For selection we used impurity importance from a random forest together with permutation importance on the validation set, and grouped the results by feature family so we could see which kinds of signals mattered rather than which individual columns. Temperature and power features dominate. After dropping everything with non-positive permutation importance we were left with 24 features.

We deliberately left out the current temperature reading and used its lagged value instead. Keeping it would let the model lean directly on the quantity the label is defined from, and we would rather have a slightly weaker model than a misleading one.

5 Splits and Models

We split chronologically by session: XXX sessions for training, XXX for validation, XXX for test, which works

Table 1: Class balance in each split.

Split	Sessions	Samples	Positives	Rate
Train	9	15,744	2,498	15.9 %
Validation	2	3,687	115	3.1 %
Test	2	3,719	108	2.9 %

Table 2: Validation F_1 after a small random search over hyperparameters.

Model	Validation F_1
Gradient boosting (chosen)	0.350
Random forest	0.314
Logistic regression	0.181
Neural network	0.106

out to roughly 70/15/15 by sample count. Preprocessing is fitted on the training sessions only and then applied unchanged to the other two.

Table 1 shows a problem we did not anticipate. The positive rate in training is about five times what it is in validation and test, because the sessions with the most stress happened to be recorded early on. So the models are selected on a much sparser regime than the one they were fitted to, and the test metrics rest on only 108 positive samples. We would not put much weight on small differences between models, given that.

We compared four models: logistic regression, random forest, gradient boosting, and a small neural network. For each we also tried class weighting, random oversampling, and SMOTE. None of these beat leaving the imbalance alone, so we dropped them. SMOTE in particular was never a good fit here: it interpolates between rows that are not adjacent in time, which invents machine states that could not physically occur.

Table 2 gives the validation scores. The trees are well ahead, which fits with what the exploration suggested: the feature space is multimodal and highly collinear, and axis-aligned splits handle that better than a linear boundary. Logistic regression suffers directly from the collinearity in Figure 4. The neural network is last, which is unsurprising with this little data. We picked gradient boosting on validation and only then looked at the test set.

6 Results

These numbers come from the 13 sessions we have now, and we expect them to move.

Our baseline is persistence: predict an event whenever the current temperature is already at or above the threshold. It is a low bar, but it is the honest one, given how autocorrelated the series is.

The accuracy is 0.97, and it means nothing: predicting "no event" every time would get almost the same score. What matters is the positive class, and there the model is weak. It finds 5 of the 108 events. It does beat persistence on F_1 (0.073 against 0.052), but not by an

Table 3: Test performance, positive class.

	Persistence	Gradient boosting
Precision	0.429	0.179
Recall	0.028	0.046
F_1	0.052	0.073
ROC-AUC	—	0.752
PR-AUC	—	0.078
Events found (of 108)	3	5

Table 4: Confusion matrix on the test sessions.

	Predicted 0	Predicted 1
Actual 0	3,588	23
Actual 1	103	5

amount anyone should be impressed by.

The AUC figures are the reason we are not discouraged. ROC-AUC of 0.75, and a PR-AUC of 0.078 against a base rate of 0.029, say the model ranks prevent samples well above random. The information is in the features. What is failing is the step from a ranking to a yes/no decision at a fixed 0.5 cutoff, which is badly suited to a class that only appears 3% of the time.

7 What We Take From This

The main lesson is that our problem right now is data, not modelling. Training and test disagree on the base rate by a factor of five, and 108 positive test samples cannot support careful comparisons between models. Collecting more sessions, especially thermally varied ones, will move these numbers more than swapping in a better classifier would.

The second point is that the horizon defines the difficulty. At one minute the task is mostly solved by inertia; at two minutes persistence collapses to 2.8% recall and the problem becomes real. The poor scores are partly a consequence of having chosen a more realistic version of the problem, even though it makes the results look worse.

Third, we have not touched the decision threshold. Everything above uses 0.5, while the AUC results suggest the model ranks well but is calibrated for a much more balanced world than the one it is in. Tuning the threshold on validation is the cheapest improvement available to us and is the first thing we plan to do.

Finally, the imputed temperatures bother us. Roughly a sixth of the label source was reconstructed, so some labels are built on values we did not observe. We have not yet checked whether the model behaves differently on imputed stretches, and we should.

8 Limitations and Next Steps

The obvious limitations: only XXX sessions, from two machines; a large gap in positive rate between training and test; test metrics resting on 108 positives; labels partly built on imputed temperature; and a hyperpa-

rameter search that only tried five configurations per model at a fixed threshold. We also selected features using unscaled trees while the final models see transformed inputs, which is untidy and should be made consistent.

What we plan to do next, roughly in order:

1. Record more stress sessions, so that events are spread across the splits instead of piling up in training.
2. Tune the decision threshold on validation and report the precision-recall curve rather than a single operating point.
3. Move to cross-validation across sessions once we have enough of them for that to be meaningful.
4. Revisit the threshold and the horizon together, once the larger dataset shows us what is actually learnable.
5. Check how the model performs on stretches where the temperature was imputed.

The pipeline itself is stable at this point. Sections 5 and 6 are the parts we expect to rewrite.